

Coded Application Services

Virtual Try-On & Measurement Feature

Technical Specification Document (Simple English)

What Is This Feature About?

This document explains how the **Virtual Try-On & Measurement Service** works. It is for merchants, including tailors, boutique owners, and market sellers, who want to:

- Show how their clothes look on different people without hiring models.
- Let customers measure themselves from their phones.
- Send order details with measurements to merchants via WhatsApp.

The feature uses **free AI tools** and **browser technology** so merchants do not need expensive software or servers.

The Two Main Parts of This Feature

Part A: Virtual Try-On Service

What it does: Takes a photo of a piece of clothing and shows it on a digital model.

How it works:

1. Merchant uploads a clean photo of clothing, such as a shirt, dress, or pair of pants.
2. Merchant picks a model style, such as tall, medium, or curvy, or uploads a specific person's photo.
3. The AI wraps the clothing onto the model realistically.
4. Merchant downloads the result and posts it on Instagram or WhatsApp.

Who uses it: Store owners creating marketing posts.

Part B: Measurement Scanner Service

What it does: Uses a phone camera to measure a customer's body.

How it works:

1. Customer visits the merchant's store on Vercel.
2. Customer clicks "Scan My Measurements."
3. Phone camera scans 33 body points, including shoulders, arms, waist, and legs.
4. Measurements are saved and added to the order.
5. When the customer sends the order via WhatsApp, measurements are included.

Who uses it: Customers ordering tailored or bespoke clothing.

What Technology Do We Need?

Free & Open Source Tools Required

A. Frontend Framework

- **Next.js** (React framework)
- Free to use.
- Hosts on Vercel using the merchant's own account.
- Handles the user interface.

B. Body Scanning Technology

- **MediaPipe** by Google.
- Free open-source library.
- Detects 33 body landmarks from a phone camera.
- Works in the browser, so no app is needed.
- Uses the phone's own processing power (GPU).
- No data is sent to Google servers during scanning.

C. AI Image Processing

- **Google Gemini 2.5 Flash (Nano Banana)**.
- Free tier available.
- Removes backgrounds from clothing photos.
- Adds clothes to model photos.
- Generates marketing images.
- API calls are free within limits.

D. Database & Backend

- **Supabase** (open-source PostgreSQL).
- Free tier sufficient for startups.
- Stores merchant data, with 20 images maximum per merchant.
- Stores customer measurements and order history.
- Provides a real-time API to feed inventory to Vercel sites.

E. Payment Processing

- **Hubtel** (Ghana-specific).
- Handles MoMo and card payments.
- Manages monthly subscriptions at GHS 75/month.
- Syncs payment status to the backend.

F. Communication Channels

- **WhatsApp API** via Twilio or a similar provider, with free tier available.
- Sends order summaries to merchants.
- Allows customers to export basket links.
- **Telegram API** is a completely free alternative sharing channel.

System Architecture (Simple Breakdown)

Layer 1: What the Customer Sees (Frontend)

```
Merchant's Vercel Website
|-- Product Gallery (AI-cleaned images)
|-- Virtual Try-On Tool
|   |-- Upload clothing photo
|   |-- Select model style
|   |-- Download result
|-- Measurement Scanner
|   |-- Camera access
|   |-- Body scanning
|   |-- Save measurements
|-- Shopping Basket & Checkout
|   |-- Add items
|   |-- Generate basket link
|   |-- Send via WhatsApp
```

Layer 2: What Happens on the Phone (Client Processing)

```
Phone (No app download needed)
|-- MediaPipe runs in browser
|   |-- Accesses phone camera
|   |-- Detects 33 body landmarks
|   |-- Calculates measurements
|-- Sends to backend for storage
|   |-- Measurements stay in customer's private order
```

Layer 3: The Powered Brain (Backend)

```
Coded Backend (Supabase)
|-- Merchant Accounts
|   |-- Subscription status
|   |-- 20 AI-cleaned images
|   |-- API keys
|-- Customer Data
|   |-- Measurements (secure)
|   |-- Order history
|   |-- Basket contents
|-- API Service
|   |-- Feeds images to Vercel
```

```
|-- Stores measurements
|-- Checks payment status
```

Layer 4: The AI Brain (Google AI)

```
Google Gemini 2.5 Flash
|-- Image Cleaning
|   |-- Remove background
|   |-- Add studio background
|   |-- Enhance product photo
|-- Try-On Processing
    |-- Drape clothing on model
    |-- Adjust fit
    |-- Generate marketing image
```

Step-by-Step: How It Works

Scenario A: Merchant Creates a Marketing Image Using Virtual Try-On

Step 1: Merchant prepares

- Logs into the Vercel website.
- Selects a product from inventory using the AI-cleaned image.
- Chooses a model style, such as “Tall Lean Model.”

Step 2: AI processes

- Google Gemini AI receives the product photo, model reference photo, and prompt template.
- Prompt template: “Drape this [clothing] realistically onto this [model] for e-commerce marketing.”
- AI generates a photo showing the clothing on the model.

Step 3: Merchant exports

- Downloads the high-resolution image.
- Posts it on Instagram.
- Includes it in a marketing carousel.

Cost: Free within Google’s free tier limits.

Scenario B: Customer Measures Themselves for a Tailored Order

Step 1: Customer prepares

- Visits the merchant’s Vercel store.
- Clicks “Get Measurements for Tailored Order.”
- Grants camera permission when the browser asks.

Step 2: Scanning happens

- Phone camera turns on.

- MediaPipe detects shoulders, arms, waist, hips, legs, inseam, and other points, for 33 points total.
- The scan takes 10–15 seconds.
- No data leaves the phone yet.

Step 3: Measurements sent

- Customer confirms measurements look correct.
- Measurements are sent to Supabase in encrypted form.
- Measurements are stored in the customer’s secure order record.

Step 4: Order checkout

- Customer adds items to basket.
- Customer clicks “Export Order.”
- Basket link is generated with measurements included.
- Customer copies the link and sends it to the merchant via WhatsApp.

Step 5: Merchant receives order

- Merchant gets a WhatsApp message with product names and prices.
- Product photos are included using AI-cleaned images.
- Customer’s measurements, including all 33 points, are included.
- Order total is included.
- Merchant fulfills the order using the measurements.

Cost to customer: Free. No app and no SMS charges are required.

Data Storage & Privacy

What We Store Permanently in Supabase

- Merchant account information.
- 20 AI-cleaned product images per merchant.
- Customer measurements for order fulfillment.
- Order history.
- Subscription status.

What We Do Not Store

- Generated marketing graphics, because merchants download them.
- API keys, because they are stored in the merchant’s browser only.
- Raw camera video, which is never uploaded.
- Try-on attempt photos, unless the merchant saves the final result.

Security

- Supabase uses encryption at rest.
- API gates all data access by subscription status.
- If a merchant does not pay, inventory is hidden and dashboard access is locked.

- Measurements are stored securely and are only visible in the order context.

Costs Breakdown

For Coded (The Platform Owner)

- **Supabase:** approximately \$0–50/month. Free tier handles hundreds of merchants.
- **Google Gemini API:** Free, up to 15 requests/minute on the Nano Banana tier.
- **Hosting:** \$0, because merchants host on their own Vercel.
- **Total:** Near-zero compute cost per merchant.

For Merchants (Monthly Subscription)

- **Starter Plan:** GHS 75/month.
- Includes 20 AI-cleaned images, try-on studio, measurement scanner, and API access.

Revenue Model

- Monthly subscription fee of GHS 75 per merchant.
- Revenue security: if subscription is not paid, the API feed stops and inventory is hidden.

Technical Requirements to Build This

Frontend Requirements (Next.js on Vercel)

Must-Have Components:

1. Product gallery display
2. Virtual try-on interface
 - Image upload form
 - Model selector dropdown
 - Result preview & download
3. Measurement scanner interface
 - MediaPipe integration
 - Camera permission handler
 - Results display
4. Shopping basket
 - Add/remove items
 - Basket link generator
5. WhatsApp/Telegram export

Must-Have Libraries (Free/Open Source):

- next.js (React framework)
- mediapipe (body scanning)
- @react-three/fiber (optional, for 3D model visualization)
- zustand or context-api (state management)
- axios or fetch (API calls)

Backend Requirements (Supabase)

Must-Have Database Tables:

1. merchants
 - id, email, subscription_status, api_key, vercel_url
2. products (max 20 per merchant)
 - id, merchant_id, name, price, image_url, stock_status
3. customers
 - id, merchant_id, email, phone
4. measurements
 - id, customer_id, landmark_data (JSON), order_id
5. orders
 - id, customer_id, products, total, measurements_id, created_at
6. subscriptions
 - id, merchant_id, status, renewal_date, payment_method

Must-Have APIs:

- GET /products/:merchant_id (fetch inventory)
- POST /measurements (store scanned data)
- POST /orders (create order with measurements)
- GET /subscription-status (check if merchant paid)
- POST /basket-link (generate shareable link)

AI Integration Requirements (Google Gemini API)

Prompt Templates Needed:

1. Image Cleaning Prompt

"Remove background from this clothing photo. Replace with clean white studio background. Keep only the garment. Output should look professional e-commerce ready."
2. Virtual Try-On Prompt

"Take this clothing item and drape it realistically onto this model photo. Adjust fit and proportions. Output high-quality marketing image."
3. Garment Detection Prompt

"What type of clothing is in this image? Extract: category, color, pattern, size hints."

Measurement Data Structure

```
{
  "measurements": {
    "landmarks": [
      {"label": "left_shoulder", "x": 0.45, "y": 0.20, "z": 0.05},
      {"label": "right_shoulder", "x": 0.55, "y": 0.20, "z": 0.05}
    ],
    "derived_measurements": {
      "shoulder_width_cm": 42,
      "chest_cm": 95,
      "waist_cm": 85,
      "hip_cm": 100,
      "inseam_cm": 78
    }
  },
}
```

```
"scan_timestamp": "2026-06-04T10:30:00Z"
  }
}
```

Step-by-Step Implementation Timeline (1 Month Accelerated)

Week 1: Foundation & Virtual Try-On Core

- Set up Supabase database schema for all tables.
- Create merchant authentication with login and signup.
- Build basic Next.js frontend structure.
- Integrate Google Gemini API.
- Build image upload interface.
- Build model selector component.
- Create prompt templates for try-on.

Week 2: Virtual Try-On Completion & Measurement Start

- Build try-on result viewer and downloader.
- Test virtual try-on end-to-end.
- Integrate MediaPipe library.
- Build camera interface.
- Implement landmark detection for 33 points.
- Create measurements display UI.

Week 3: Measurement Completion & Shopping Integration

- Store measurements in the database.
- Build shopping basket UI.
- Set up API routes for inventory feed.
- Create basket link generator.
- Integrate basic WhatsApp API setup.
- Integrate basic Telegram API setup.

Week 4: Payment, Security & Launch

- Integrate Hubtel payments.
- Build subscription gatekeeper for access control.
- Implement API access controls.
- Test revenue security using paid and unpaid states.
- Conduct end-to-end testing of all features.
- Optimize performance.
- Conduct a security audit.
- Launch beta with 5–10 merchants.

Success Metrics

For Merchants

- **Time to create marketing image:** less than 2 minutes.
- **Accuracy of measurements:** within 2–5cm margin.
- **Subscription retention:** greater than 80% monthly.
- **Orders via measurement link:** track percentage of checkout traffic.

For Coded

- **API uptime:** greater than 99.5%.
- **Image processing speed:** less than 30 seconds per image.
- **Customer acquisition cost:** less than GHS 50.
- **Monthly revenue:** merchants multiplied by GHS 75.

Open Source & Free Resources Used

Resource	Purpose	License	Cost
Next.js	Frontend framework	MIT	Free
React	UI library	MIT	Free
MediaPipe	Body scanning	Apache 2.0	Free
Supabase	Database & API	Open Source	Free tier / \$25/mo
Google Gemini	Image AI	Proprietary API	Free tier (15 req/min)
Vercel	Hosting	Proprietary	Free tier / \$20/mo per merchant
TailwindCSS	Styling	MIT	Free
Axios	HTTP client	MIT	Free
Zustand	State management	MIT	Free

FAQ (Frequently Asked Questions)

Q: Do customers need to download an app?

A: No. Everything runs in the phone's web browser. No app download is needed.

Q: Is this secure for storing measurements?

A: Yes. Measurements are encrypted in Supabase and only accessible via API authentication.

Q: What happens if the merchant does not pay?

A: The subscription gateway cuts off API access. Inventory hides from their Vercel site. Dashboard

access freezes.

Q: How many images can a merchant store?

A: 20 images maximum. This keeps storage costs low and encourages merchants to update their inventory regularly.

Q: Can customers share their measurements with friends?

A: Yes. The basket link includes measurements, so they can send it via WhatsApp to friends or to the merchant.

Q: What if Google Gemini API changes pricing?

A: This is a business risk. Coded monitors API costs. An alternative is to build an open-source image processing pipeline locally.

Known Limitations & Risks

1. **MediaPipe Accuracy:** Body scanning may have a $\pm 5\text{cm}$ margin of error. It is not suitable for extreme custom fits, but it is good enough for standard tailoring.
2. **Try-On AI Limitations:** AI may not perfectly handle all garment types, such as very flowing dresses or structured jackets. Human review is recommended.
3. **Image Cleanup Dependence:** Feature quality depends heavily on Google's AI quality. There is no backup if Google degrades service.
4. **20-Image Limit:** Merchants with large catalogs will hit the limit and may need a pricing tier upgrade later.
5. **Internet Dependency:** All features require an internet connection and may work poorly on 2G networks.

Next Steps

1. **Week 1:** Share this document with the development team.
2. **Week 1–2:** Set up the development environment, including Supabase project, Google API key, and Next.js repo.
3. **Week 2:** Begin Phase 1 implementation.
4. **Ongoing:** Hold weekly progress reviews with stakeholders.

Document Version: 1.0

Last Updated: June 4, 2026

Prepared For: Coded Application Services Development Team